23BEC0503

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

SCHOOL OF ELECTRONICS ENGINEERING

Fall Semester 2025-2026

BECE406E – FPGA Based System Design

TASK 3 – Design and Implementation of Memory modelling

SANJAY ELAVARASAN KARTHIKEYAN

23BEC0503

Aim:

Create a text file which contains Mod 50 counter values. Read the content of the text file and store in single port RAM. Create a blank text file and write the Mod 50 counter values. Compare both files and display whether values are matched or not. Attach screenshots of code, simulation, both text files.

Software Details:

- Functional Simulation: Modelsim
- Synthesis & Implementation: Intel Quartus Prime 23.1

Hardware Details:

- Family: Cyclone IV E
- Package: FBGA
- Pin count: 780
- Core speed grade – 7
- Device - EP4CE115F29C7

23BEC0503

RTL Code for Single Port RAM:

C:/Users/SANJAY ELAVARASAN/Desktop/FPGA/single_port_ram.v - Default *

```
//23BEC0503 - SANJAY ELAVARASAN KARTHIKEYAN
module single_port_ram(clk, we, addr, din, dout);

input clk;
input we;
input [5:0] addr;
input [7:0] din;
output reg [7:0] dout;

reg [7:0] ram [49:0];

always @(posedge clk) begin
    if (we)
        ram[addr] <= din;
    else
        dout <= ram[addr];
end

endmodule
```

23BEC0503

RTL Code for MOD 50 Counter:

C:/Users/SANJAY ELAVARASAN/Desktop/FPGA/mod_50_counter.v - Default

```
//23BEC0503 - SANJAY ELAVARASN KARTHIKEYAN
module mod50(clk, rst, count);

input clk, rst;
output reg [5:0] count;

always @(posedge clk or posedge rst)
begin
    if (rst)
        count <= 0;
    else
    begin
        if (count == 6'd49)
            count <= 0;
        else
            count <= count + 1;
    end
end

endmodule
```

single_port_ram.v * | mod_50_counter.v

23BEC0503

RTL Code for Testbench :

```
//23BEC0503 - SANJAY ELAVARASAN KARTHIKEYAN
module tb_mod50_ram();

reg clk, rst;
wire [5:0] count;

reg [7:0] mem [49:0];
reg [7:0] written [49:0];

integer i, mismatch;
integer file;

reg we;
reg [5:0] ram_addr;
reg [7:0] din;
wire [7:0] dout;

mod50 dut1(clk, rst, count);

single_port_ram ram1(
    .clk(clk),
    .we(we),
    .addr(ram_addr),
    .din(din),
    .dout(dout)
);

initial
    $readmemh("input_23BEC0503.txt", mem, 0, 49);

always
    #5 clk = ~clk;

initial begin
    clk = 0;
    rst = 1;
    we = 0;
    ram_addr = 0;
    din = 0;

    #10 rst = 0;

    $display("------------------------------------------------------------");
    $display("STEP 1: Writing input_23BEC0503.txt values into RAM");
    $display("------------------------------------------------------------");

    we = 1;
    for (i = 0; i < 50; i = i + 1) begin
        @(posedge clk);
        #1;
        ram_addr = i[5:0];
        din = mem[i];
        $display("WRITE  addr=%0d  data=%0h", i, mem[i]);
    end

    @(posedge clk);
    #1;
    we = 0;
```

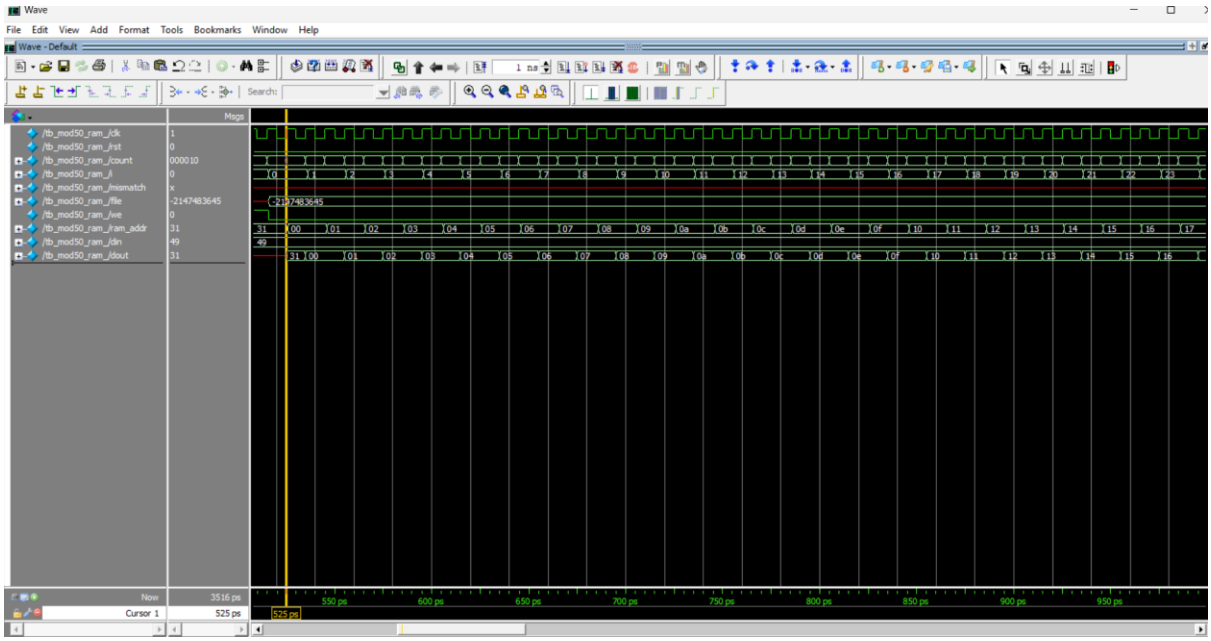
23BEC0503

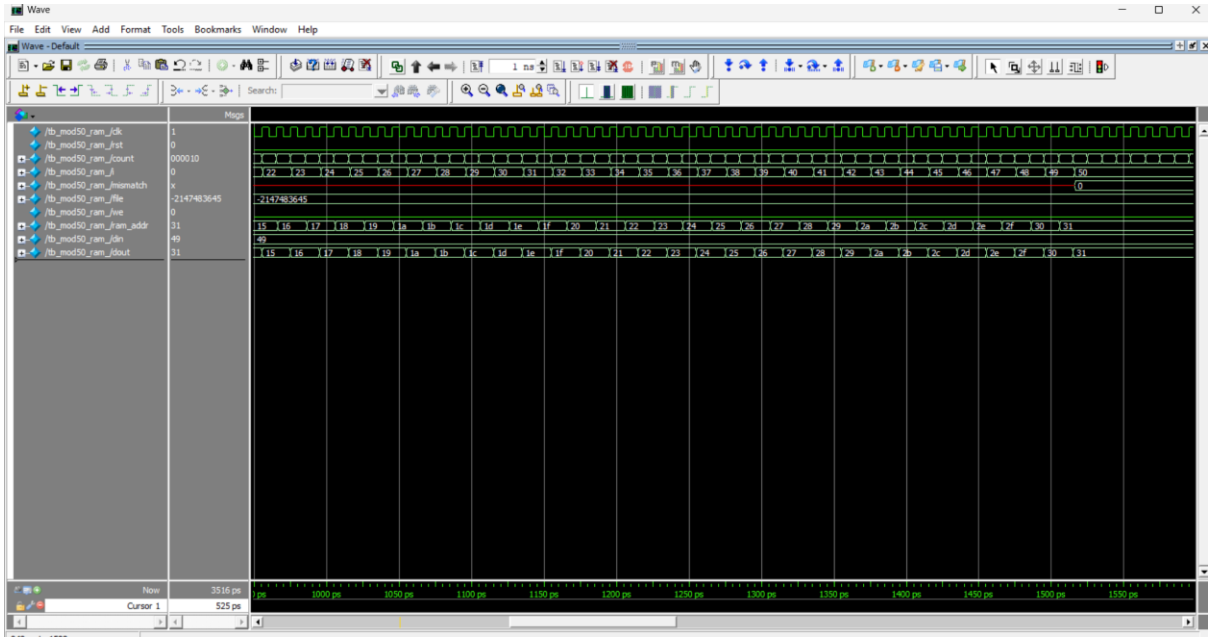
```
    $display("------------------------------------------------------------");
    $display("STEP 2: Reading from RAM -> write_50_23BEC0503.txt");
    $display("------------------------------------------------------------");

    file = $fopen("write_50_23BEC0503.txt", "w");
    if (file == 0) begin
        $display("ERROR: Cannot open write_50_23BEC0503.txt");
        $finish;
    end

    for (i = 0; i < 50; i = i + 1) begin
        @(posedge clk);
        #1;
        ram_addr = i[5:0];

        @(posedge clk);
        #1;
        written[i] = dout;
        $fwrite(file, "%0h\n", dout);
        $display("READ   addr=%0d  data=%0h", i, dout);
    end

    $fclose(file);

    $display("------------------------------------------------------------");
    $display("STEP 3: Comparing input_23BEC0503.txt vs write_50_23BEC0503.txt");
    $display(" Addr | input_23BEC0503.txt | write_50_23BEC0503.txt | Status");
    $display("------------------------------------------------------------");

    mismatch = 0;
    for (i = 0; i < 50; i = i + 1) begin
        if (mem[i] == written[i]) begin
            $display(" [%2d]   %0h                %0h             MATCH",
                     i, mem[i], written[i]);
        end
        else begin
            $display(" [%2d]   %0h                %0h             MISMATCH",
                     i, mem[i], written[i]);
            mismatch = mismatch + 1;
        end
    end

    $display("------------------------------------------------------------");
    if (mismatch == 0)
```

```
    if (mismatch == 0)
        $display("RESULT: All 50 values MATCHED successfully!");
    else
        $display("RESULT: %0d MISMATCH(es) found!", mismatch);
    $display("------------------------------------------------------------");

    $finish;
end

endmodule
```

23BEC0503

FUNCTIONAL SIMULATION



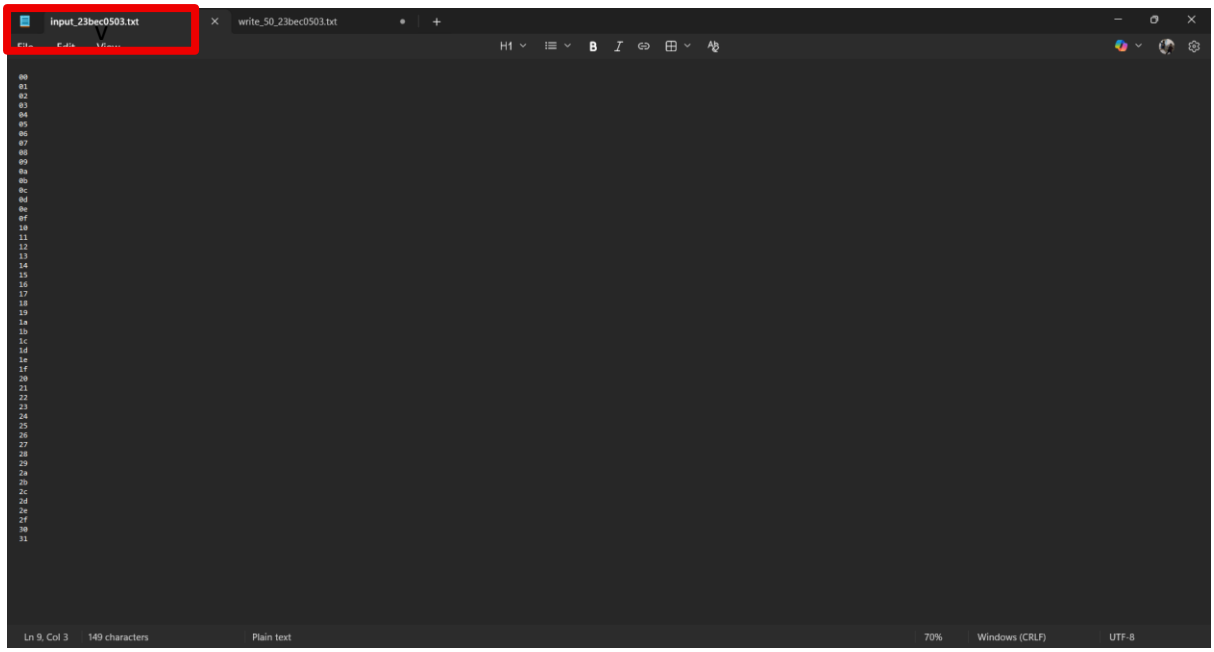


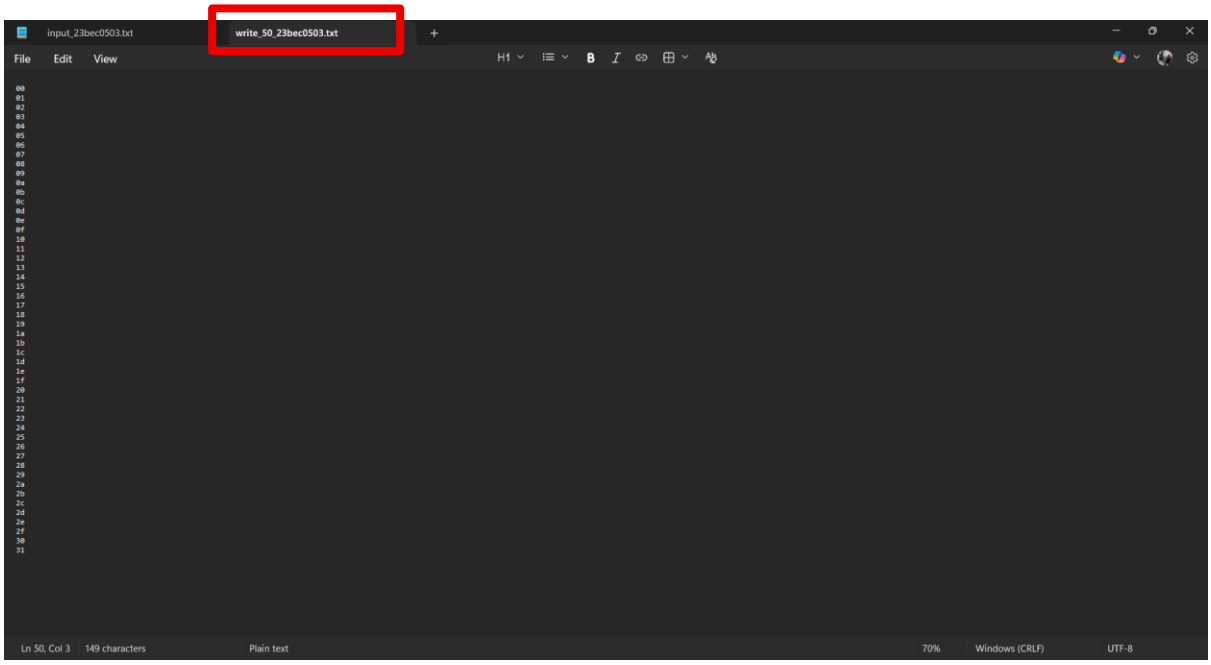
23BEC0503

```
# ----------------------------------------------------
#   Addr | input.txt | write_50.txt | Status
#   -----|-----------|--------------|--------
#   [ 0]   00          00           MATCH
#   [ 1]   01          01           MATCH
#   [ 2]   02          02           MATCH
#   [ 3]   03          03           MATCH
#   [ 4]   04          04           MATCH
#   [ 5]   05          05           MATCH
#   [ 6]   06          06           MATCH
#   [ 7]   07          07           MATCH
#   [ 8]   08          08           MATCH
#   [ 9]   09          09           MATCH
#   [10]   0a          0a           MATCH
#   [11]   0b          0b           MATCH
#   [12]   0c          0c           MATCH
#   [13]   0d          0d           MATCH
#   [14]   0e          0e           MATCH
#   [15]   0f          0f           MATCH
#   [16]   10          10           MATCH
#   [17]   11          11           MATCH
#   [18]   12          12           MATCH
#   [19]   13          13           MATCH
#   [20]   14          14           MATCH
#   [21]   15          15           MATCH
#   [22]   16          16           MATCH
#   [23]   17          17           MATCH
#   [24]   18          18           MATCH
#   [25]   19          19           MATCH
#   [26]   1a          1a           MATCH
#   [27]   1b          1b           MATCH
#   [28]   1c          1c           MATCH
#   [29]   1d          1d           MATCH
#   [30]   1e          1e           MATCH
#   [31]   1f          1f           MATCH
#   [32]   20          20           MATCH
#   [33]   21          21           MATCH
#   [34]   22          22           MATCH
#   [35]   23          23           MATCH
#   [36]   24          24           MATCH
#   [37]   25          25           MATCH
#   [38]   26          26           MATCH
#   [39]   27          27           MATCH
#   [40]   28          28           MATCH
#   [41]   29          29           MATCH
```

```
#   [41]   29          29           MATCH
#   [42]   2a          2a           MATCH
#   [43]   2b          2b           MATCH
#   [44]   2c          2c           MATCH
#   [45]   2d          2d           MATCH
#   [46]   2e          2e           MATCH
#   [47]   2f          2f           MATCH
#   [48]   30          30           MATCH
#   [49]   31          31           MATCH
# ----------------------------------------------------
#   RESULT: All 50 values MATCHED successfully!
# ----------------------------------------------------
# ** Note: $finish    : C:/Users/SANJAY ELAVARASAN/Desktop/FPGA_DA_3/tb_mod50_ram_.v(119)
#    Time: 1516 ps  Iteration: 0  Instance: /tb_mod50_ram_
# 1
# Break in Module tb_mod50_ram_ at C:/Users/SANJAY ELAVARASAN/Desktop/FPGA_DA_3/tb_mod50_ram_.v line 119
run
VSIM 28> run

VSIM 28>
```

23BEC0503

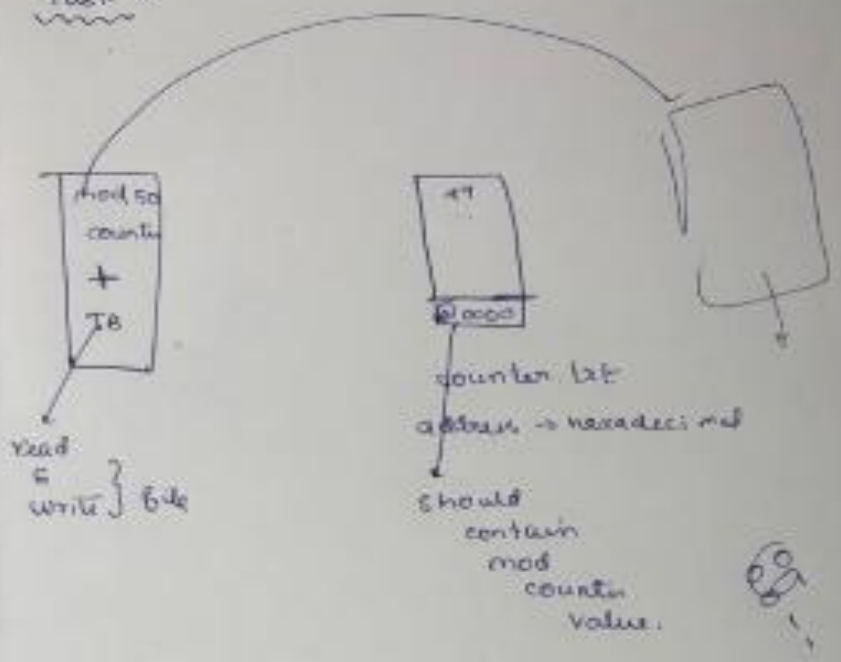

input_23bec0503.txt
write_50_23bec0503.txt
Ln 9, Col 3
149 characters
Plain text
70%
Windows (CRLF)
UTF-8


input_23bec0503.txt
write_50_23bec0503.txt
File
Edit
View
Ln 50, Col 3
149 characters
Plain text
70%
Windows (CRLF)
UTF-8

23BEC0503

Attestation copy

Task 3



$ read mem h ("filename", memory name, start addr, end addr)

must ← optional →

Write

```
file = fopen("filename");
fwrite (file, "%d \n", count);
fdisplay (file);
fclose (file);
```

KIOP R/W data
4/2
23BEC0503
SANJAY ELAVARASAN
KARTHIKEYAN

Aim:

Generate a dual port RAM IP using Quartus and verify the functionality using testbench in Modelsim. Attach the RTL view of RAM and Screenshots of code and outputs.

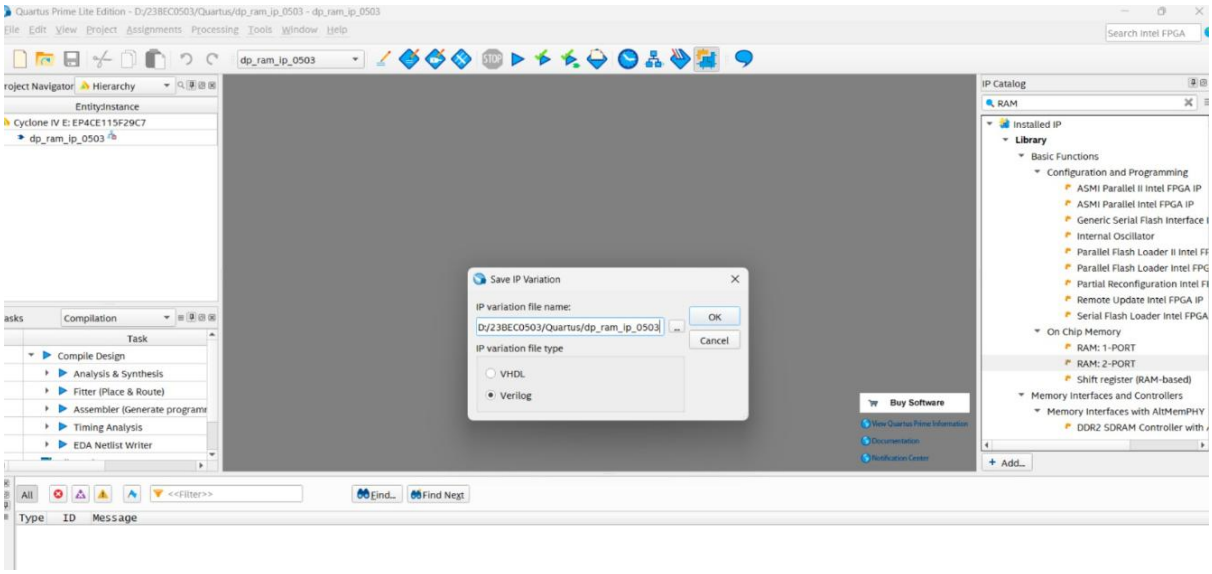
Software Details:

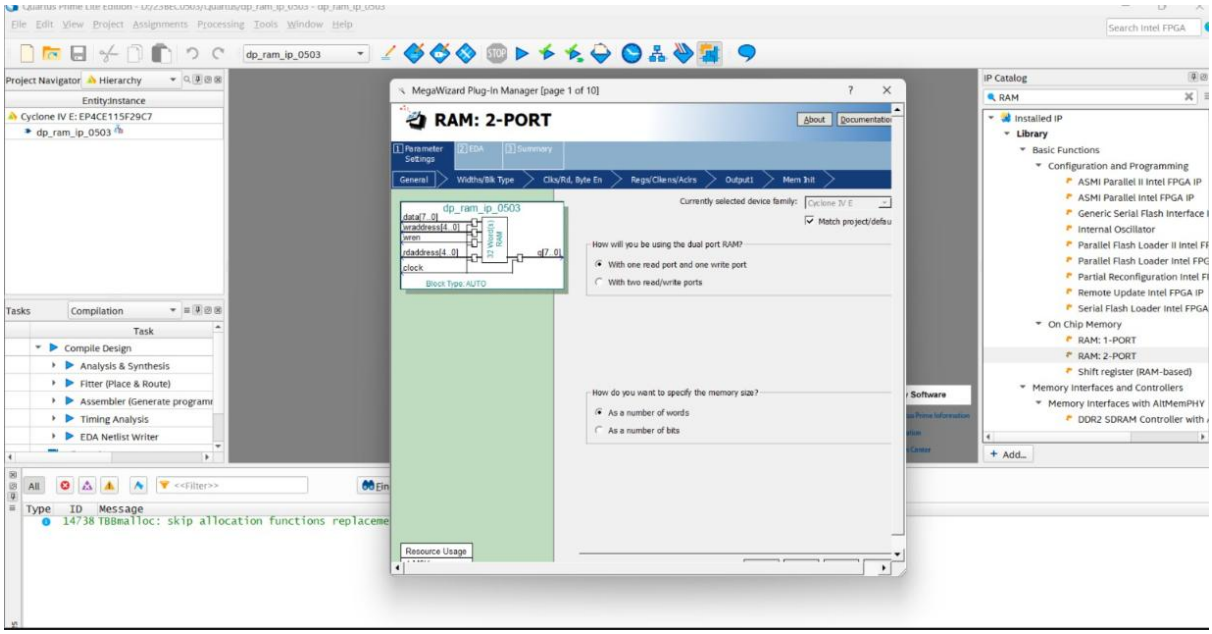
- Functional Simulation: Modelsim
- Synthesis & Implementation: Intel Quartus Prime 23.1

Hardware Details:

- Family: Cyclone IV E
- Package: FBGA
- Pin count: 780Core speed grade – 7
- Device - EP4CE115F29C7

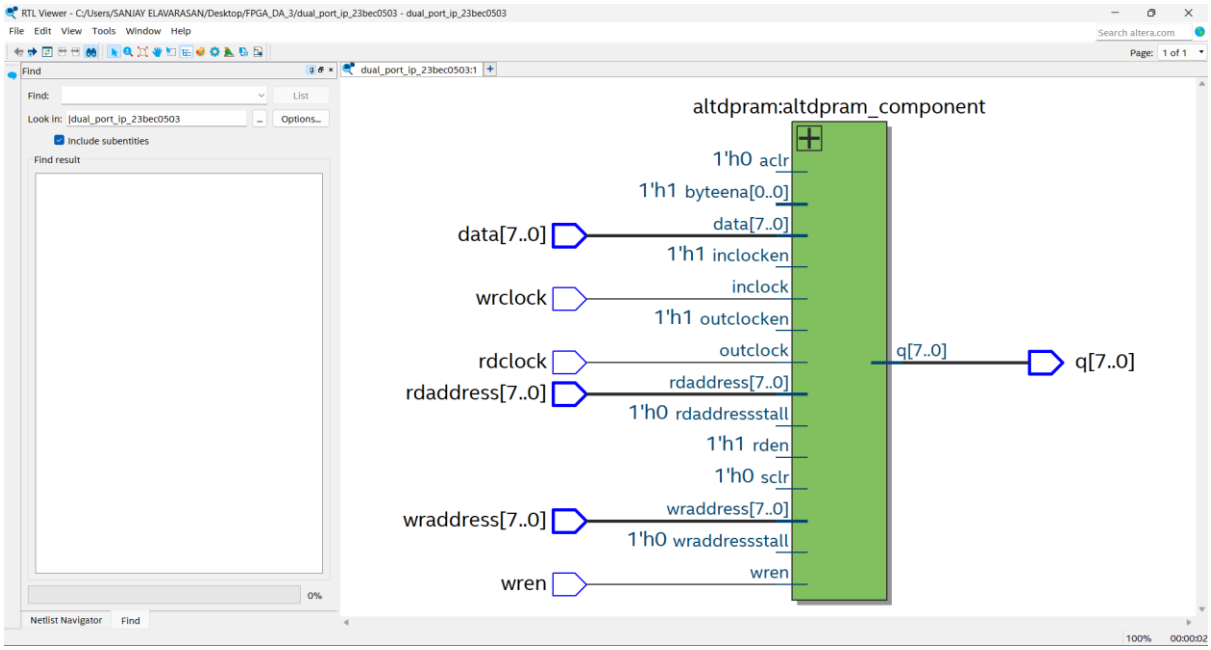
23BEC0503

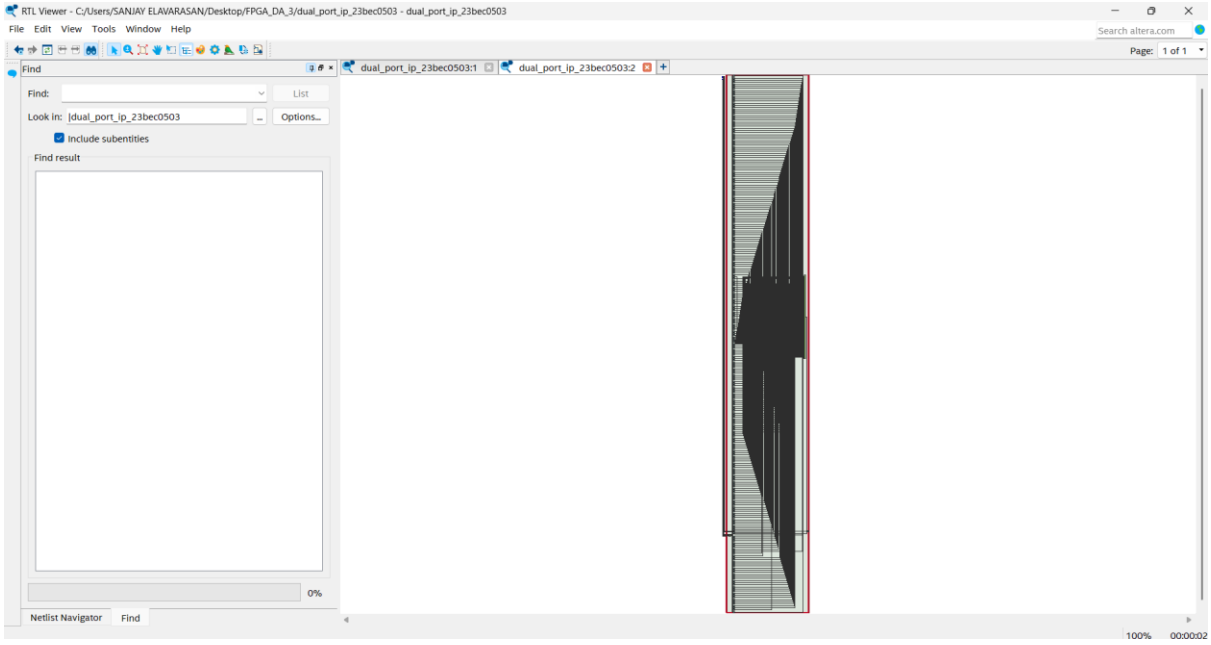

Quartus Prime Lite Edition - D:/23BEC0503/Quartus/dp_ram_ip_0503 - dp_ram_ip_0503
File Edit View Project Assignments Processing Tools Window Help
dp_ram_ip_0503
Project Navigator Hierarchy
Entity:Instance
Cyclone IV E: EP4CE115F29C7
dp_ram_ip_0503
Tasks Compilation
Task
Compile Design
Analysis & Synthesis
Fitter (Place & Route)
Assembler (Generate programm
Timing Analysis
EDA Netlist Writer
Save IP Variation
IP variation file name:
D:/23BEC0503/Quartus/dp_ram_ip_0503
OK
Cancel
IP variation file type
VHDL
Verilog
Buy Software
IP Catalog
RAM
Installed IP
Library
Basic Functions
Configuration and Programming
ASMI Parallel II Intel FPGA IP
ASMI Parallel Intel FPGA IP
Generic Serial Flash Interface
Internal Oscillator
Parallel Flash Loader II Intel FP
Parallel Flash Loader Intel FPG
Partial Reconfiguration Intel F
Remote Update Intel FPGA IP
Serial Flash Loader Intel FPGA
On Chip Memory
RAM: 1-PORT
RAM: 2-PORT
Shift register (RAM-based)
Memory Interfaces and Controllers
Memory Interfaces with AltMemPHY
DDR2 SDRAM Controller with
Add...
All
<<Filter>>
Find...
Find Next
Type ID Message


File Edit View Project Assignments Processing Tools Window Help
dp_ram_ip_0503
Project Navigator Hierarchy
Entity:Instance
Cyclone IV E: EP4CE115F29C7
dp_ram_ip_0503
Tasks Compilation
Task
Compile Design
Analysis & Synthesis
Fitter (Place & Route)
Assembler (Generate programm
Timing Analysis
EDA Netlist Writer
MegaWizard Plug-In Manager [page 1 of 10]
RAM: 2-PORT
About
Documentation
1 Parameter Settings
2 EDA
3 Summary
General
Widths/Blk Type
Clks/Rd, Byte En
Regs/Clkens/Aclrs
Outputs
Mem Init
dp_ram_ip_0503
data[7..0]
wraddress[4..0]
wren
rdaddress[4..0]
clock
32 Words(s) RAM
q[7..0]
Block Type: AUTO
Currently selected device family:
Cyclone IV E
Match project/defau
How will you be using the dual port RAM?
With one read port and one write port
With two read/write ports
How do you want to specify the memory size?
As a number of words
As a number of bits
Resource Usage
IP Catalog
RAM
Installed IP
Library
Basic Functions
Configuration and Programming
ASMI Parallel II Intel FPGA IP
ASMI Parallel Intel FPGA IP
Generic Serial Flash Interface
Internal Oscillator
Parallel Flash Loader II Intel FP
Parallel Flash Loader Intel FPG
Partial Reconfiguration Intel F
Remote Update Intel FPGA IP
Serial Flash Loader Intel FPGA
On Chip Memory
RAM: 1-PORT
RAM: 2-PORT
Shift register (RAM-based)
Memory Interfaces and Controllers
Memory Interfaces with AltMemPHY
DDR2 SDRAM Controller with
Add...
All
<<Filter>>
Type ID Message
14738 TBBmalloc: skip allocation functions replaceme

23BEC0503

RTL VIEW :

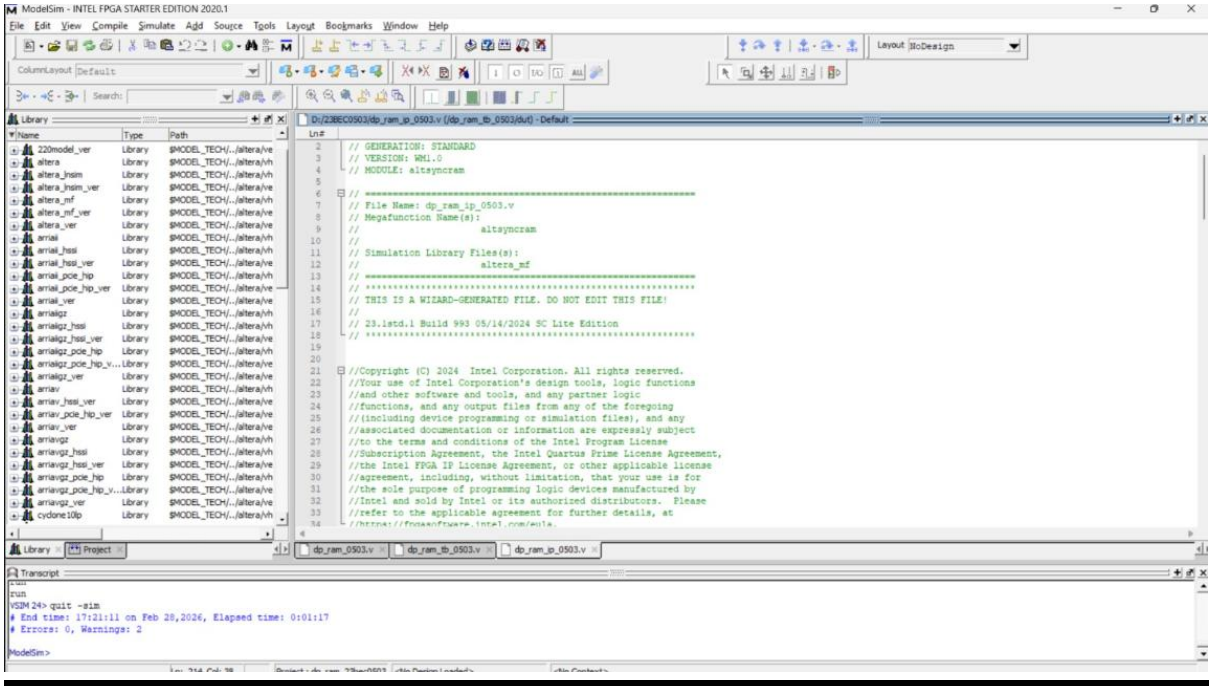


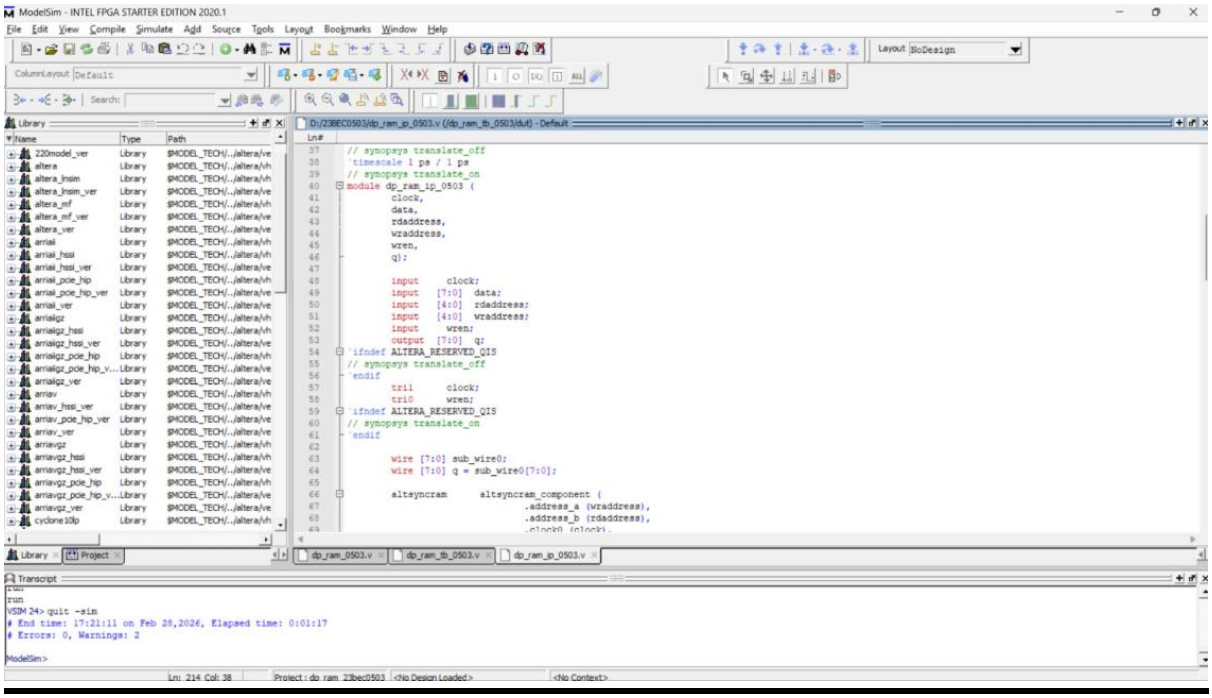


23BEC0503

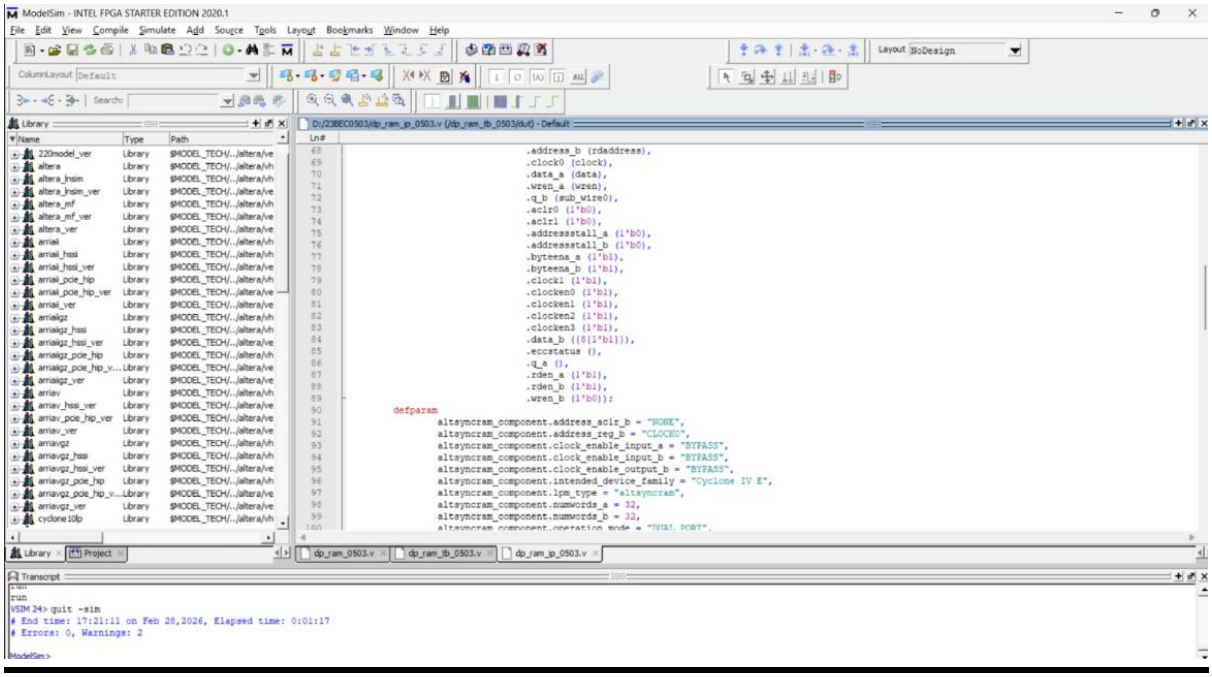
RTL CODE AND TESTBENCH CODE:

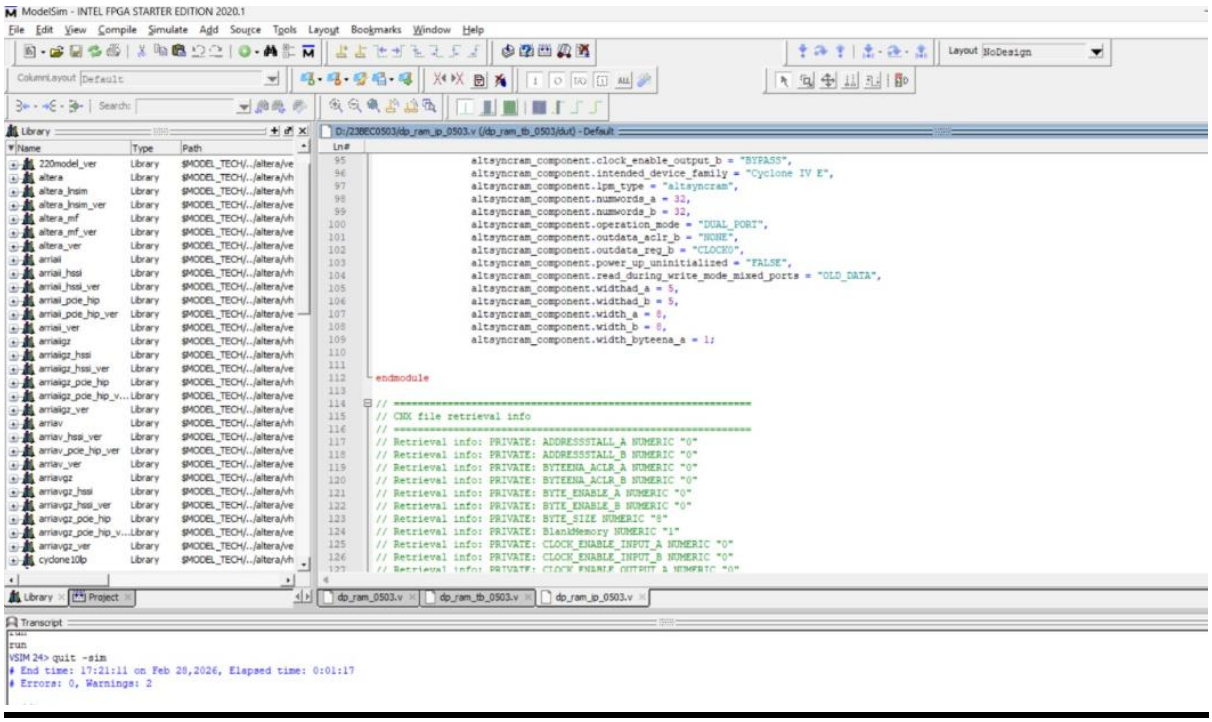
RTL :





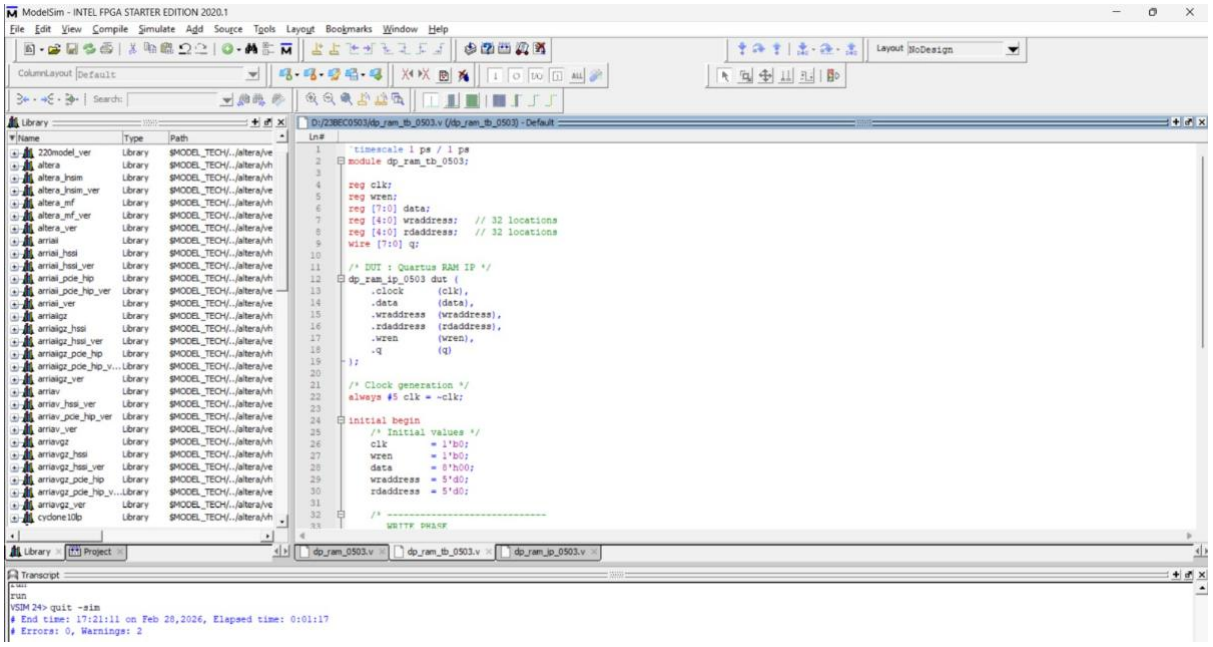
23BEC0503

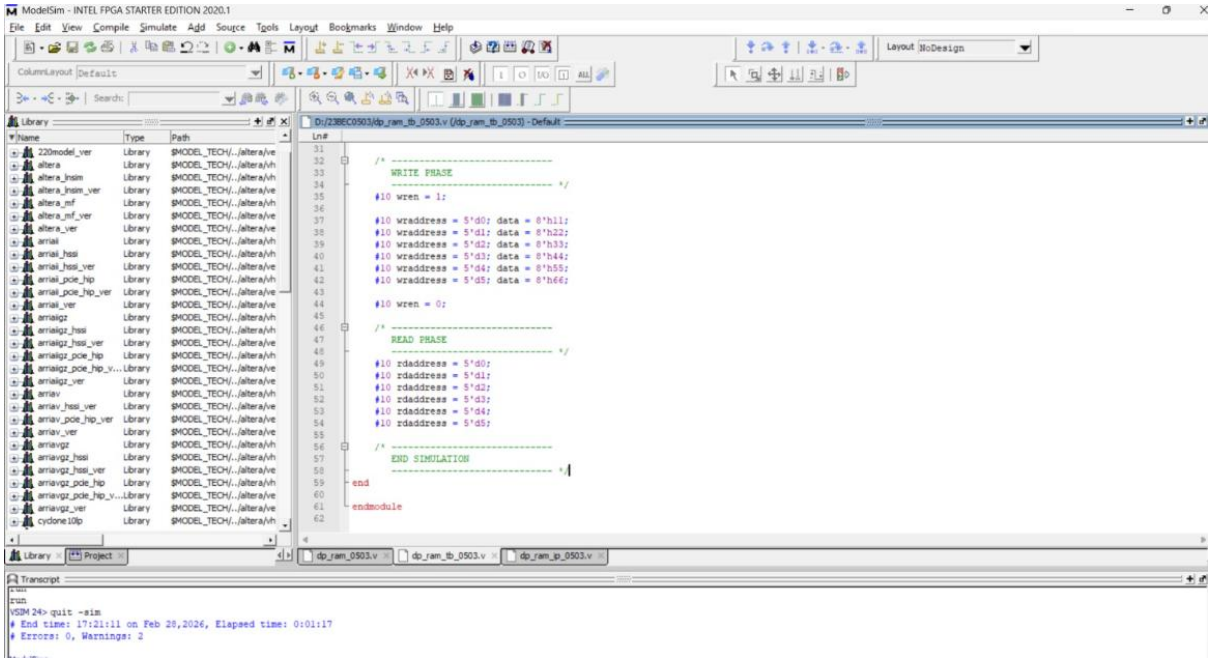

ModelSim - INTEL FPGA STARTER EDITION 2020.1
File Edit View Compile Simulate Add Source Tools Layout Bookmarks Window Help
D:/23BEC0503/dp_ram_ip_0503.v (/dp_ram_tb_0503/dut) - Default
.address_b (rdaddress),
.clock0 (clock),
.data_a (data),
.wren_a (wren),
.q_b (sub_wire0),
.aclr0 (1'b0),
.aclr1 (1'b0),
.addressstall_a (1'b0),
.addressstall_b (1'b0),
.byteena_a (1'b1),
.byteena_b (1'b1),
.clock1 (1'b1),
.clocken0 (1'b1),
.clocken1 (1'b1),
.clocken2 (1'b1),
.clocken3 (1'b1),
.data_b ({8{1'b1}}),
.eccstatus (),
.q_a (),
.rden_a (1'b1),
.rden_b (1'b1),
.wren_b (1'b0));
defparam
altsyncram_component.address_aclr_b = "NONE",
altsyncram_component.address_reg_b = "CLOCK0",
altsyncram_component.clock_enable_input_a = "BYPASS",
altsyncram_component.clock_enable_input_b = "BYPASS",
altsyncram_component.clock_enable_output_b = "BYPASS",
altsyncram_component.intended_device_family = "Cyclone IV E",
altsyncram_component.lpm_type = "altsyncram",
altsyncram_component.numwords_a = 32,
altsyncram_component.numwords_b = 32,
Transcript
run
VSIM 24> quit -sim
End time: 17:21:11 on Feb 28,2026, Elapsed time: 0:01:17
Errors: 0, Warnings: 2


ModelSim - INTEL FPGA STARTER EDITION 2020.1
File Edit View Compile Simulate Add Source Tools Layout Bookmarks Window Help
D:/23BEC0503/dp_ram_ip_0503.v (/dp_ram_tb_0503/dut) - Default
altsyncram_component.clock_enable_output_b = "BYPASS",
altsyncram_component.intended_device_family = "Cyclone IV E",
altsyncram_component.lpm_type = "altsyncram",
altsyncram_component.numwords_a = 32,
altsyncram_component.numwords_b = 32,
altsyncram_component.operation_mode = "DUAL_PORT",
altsyncram_component.outdata_aclr_b = "NONE",
altsyncram_component.outdata_reg_b = "CLOCK0",
altsyncram_component.power_up_uninitialized = "FALSE",
altsyncram_component.read_during_write_mode_mixed_ports = "OLD_DATA",
altsyncram_component.widthad_a = 5,
altsyncram_component.widthad_b = 5,
altsyncram_component.width_a = 8,
altsyncram_component.width_b = 8,
altsyncram_component.width_byteena_a = 1;
endmodule
// CHX file retrieval info
// Retrieval info: PRIVATE: ADDRESSSTALL_A NUMERIC "0"
// Retrieval info: PRIVATE: ADDRESSSTALL_B NUMERIC "0"
// Retrieval info: PRIVATE: BYTEENA_ACLR_A NUMERIC "0"
// Retrieval info: PRIVATE: BYTEENA_ACLR_B NUMERIC "0"
// Retrieval info: PRIVATE: BYTE_ENABLE_A NUMERIC "0"
// Retrieval info: PRIVATE: BYTE_ENABLE_B NUMERIC "0"
// Retrieval info: PRIVATE: BYTE_SIZE NUMERIC "8"
// Retrieval info: PRIVATE: BlankMemory NUMERIC "1"
// Retrieval info: PRIVATE: CLOCK_ENABLE_INPUT_A NUMERIC "0"
// Retrieval info: PRIVATE: CLOCK_ENABLE_INPUT_B NUMERIC "0"
Transcript
run
VSIM 24> quit -sim
End time: 17:21:11 on Feb 28,2026, Elapsed time: 0:01:17
Errors: 0, Warnings: 2

23BEC0503

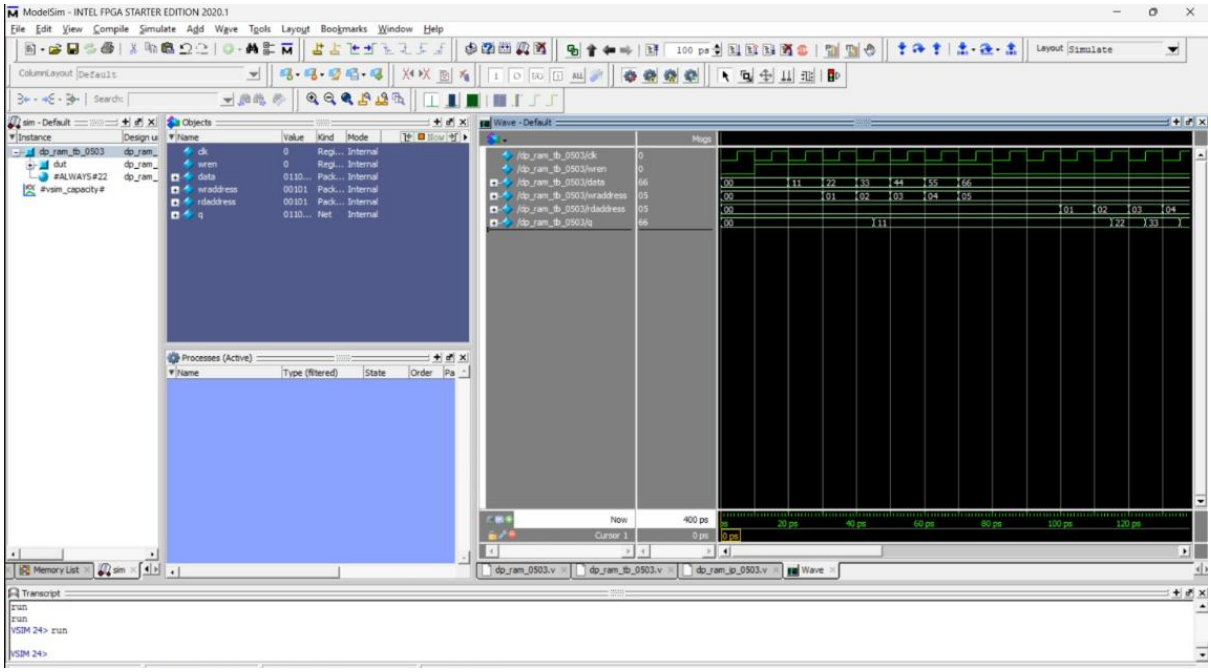
TESTBENCH :





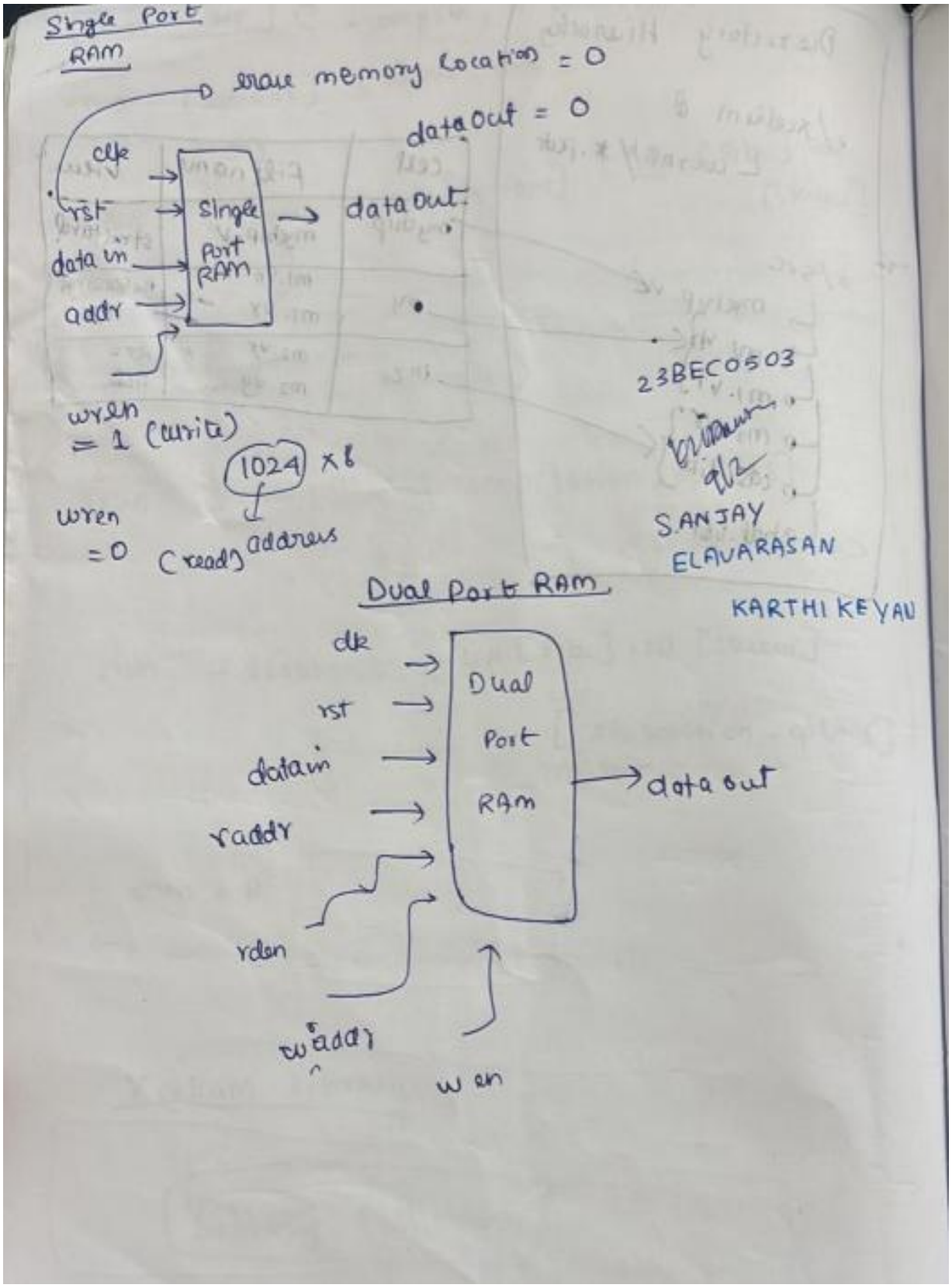
23BEC0503

FUNCTIONAL SIMULATION:



23BEC0503

Attestation copy :



INFERENCE :

The experiment verifies:

- Correct implementation of synchronous single-port and dual-port RAM.
- Understanding of clocked memory behavior.
- Proper handling of registered outputs in IP-based memory.
- Successful simulation and functional verification using ModelSim.

The design functions as expected without errors, confirming accurate memory read/write operations and proper integration of IP cores.